

TP 8 (programmation) - Snake.

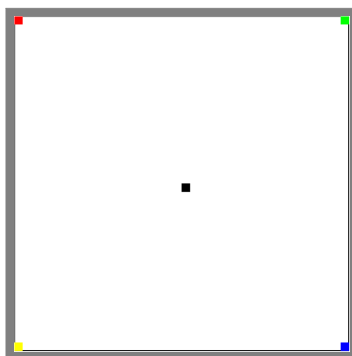
Le but du TP est de réaliser le jeu *serpent*. Dans ce jeu, on contrôle un serpent avec les flèches du clavier. Pour gagner, le serpent doit manger toutes les pommes. On perd si le serpent sort de la fenêtre ou s'il se mord la queue. Le résultat est donné dans le programme `snake` (qui fonctionne sous Linux, pensez à changer les permissions).

Récupérez le programme `snake_init.py` (et la bibliothèque graphique `fltk`, si vous ne l'avez pas déjà). Le programme contient quelques fonctions déjà écrites et d'autres fonctions que vous devrez compléter.

On découpe la fenêtre de 400x400 en carrés de 10x10. On obtient ainsi 40x40 cases. Dans le programme, une case est représentée par un **couple**. La case en haut à gauche est représentée par (0, 0), la case en bas à gauche par (0, 39), ...

La fonction `affiche_case` affiche une case dans une couleur donnée. Par exemple,

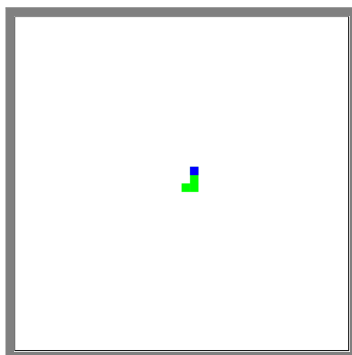
```
affiche_case((0, 0), 'red')
affiche_case((39, 0), 'green')
affiche_case((0, 39), 'yellow')
affiche_case((39, 39), 'blue')
affiche_case((20, 20), 'black')
```



Un serpent est une liste de cases, **la première étant la queue et la dernière étant la tête**. Par exemple, la liste de cases :

```
serpent = [(20, 20), (21, 20), (21, 19), (21, 18)]
affiche_serpent(serpent)
```

correspondra au serpent ci-dessous :

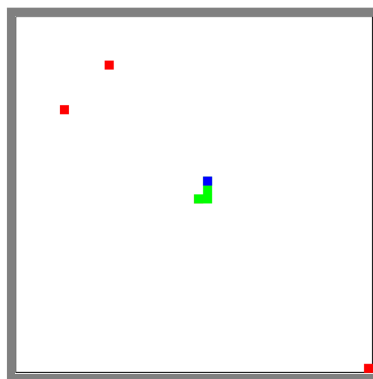


Exercice 1 (Snake)

- (a) La fonction `affiche_case` fournie dans le fichier `snake_init.py` est écrite pour un serpent découpé en carrés d'exactement 10x10. Expliquez comment elle fonctionne et modifiez-la pour qu'elle continue de fonctionner en utilisant la constante `taille_case` définie en début de fichier.
- (b) Écrivez les fonctions `affiche_cases` et `affiche_serpent`. La description de ces fonctions est en commentaires dans le fichier.
- (c) Testez vos fonctions avec le programme ci-dessous :

```
affiche_cases([(10, 5), (20, 20), (39, 39), (5, 10)], 'red')
serpent = [(20, 20), (21, 20), (21, 19), (21, 18)]
affiche_serpent(serpent)
```

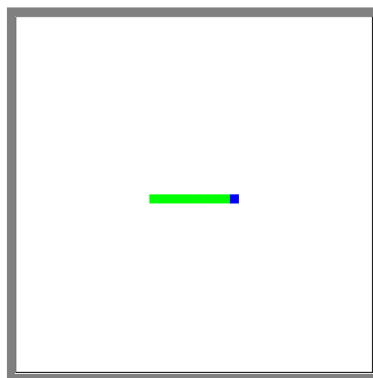
Vous devez obtenir :



- (d) Écrivez la fonction `nouveau_serpent`.
- (e) Testez la fonction avec le programme ci-dessous :

```
serpent = nouveau_serpent(10)
affiche_serpent(serpent)
```

Vous devez obtenir :

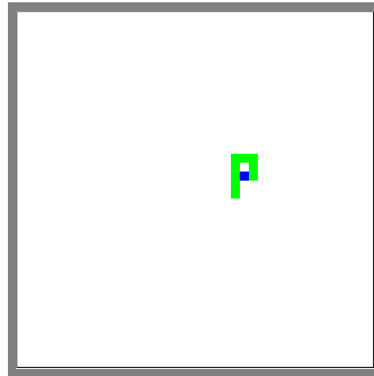


- (f) Écrivez la fonction `deplace_serpent`. La fonction prend un serpent et une direction et **renvoie** le serpent après un déplacement d'un pas dans cette direction. La direction est une chaîne de caractères qui vaut `'droite'`, `'gauche'`, `'haut'` ou `'bas'`.
- (g) Testez la fonction avec le programme ci-dessous :

```
serpent = nouveau_serpent(10)
serpent = deplace_serpent(serpent, 'haut')
serpent = deplace_serpent(serpent, 'haut')
serpent = deplace_serpent(serpent, 'haut')
```

```
serpent = deplace_serpent(serpent, 'haut')
serpent = deplace_serpent(serpent, 'droite')
serpent = deplace_serpent(serpent, 'droite')
serpent = deplace_serpent(serpent, 'bas')
serpent = deplace_serpent(serpent, 'bas')
serpent = deplace_serpent(serpent, 'gauche')
affiche_serpent(serpent)
```

Vous devez obtenir :



- (h) Écrivez les fonctions `est_sorti` et `a_mordu`.
 (i) Testez les fonctions avec le programme suivant :

```
serpent1 = nouveau_serpent(30)
serpent2 = serpent1*1
serpent2 = deplace_serpent(serpent2, 'droite')
serpent2 = deplace_serpent(serpent2, 'droite')
serpent2 = deplace_serpent(serpent2, 'droite')
serpent2 = deplace_serpent(serpent2, 'droite')
serpent2 = deplace_serpent(serpent2, 'droite')
serpent2 = deplace_serpent(serpent2, 'droite')
serpent3 = list(serpent1)
serpent3 = deplace_serpent(serpent3, 'haut')
serpent3 = deplace_serpent(serpent3, 'haut')
serpent3 = deplace_serpent(serpent3, 'gauche')
serpent3 = deplace_serpent(serpent3, 'gauche')
serpent3 = deplace_serpent(serpent3, 'bas')
serpent3 = deplace_serpent(serpent3, 'bas')

print('Serpent 1 :', est_sorti(serpent1), a_mordu(serpent1))
print('Serpent 2 :', est_sorti(serpent2), a_mordu(serpent2))
print('Serpent 3 :', est_sorti(serpent3), a_mordu(serpent3))
```

Vous devez obtenir :

```
Serpent 1 : False False
Serpent 2 : True False
Serpent 3 : False True
```

- (j) Nous allons maintenant écrire la boucle principale du jeu. Voici le code que vous devez compléter pour que votre serpent avance...

```
if __name__ == '__main__':
    cree_fenetre(400, 400)
```

```
ready()

### creer et afficher le serpent ###

direction = "droite"
temps = 0

while True:
    direction = mise_a_jour_direction(direction)
    if temps % 1800 == 0: # pour ralentir le serpent
        efface_tout()

    ### deplacer et afficher le serpent ###

    temps = temps + 1
    mise_a_jour()

fermeFenetre()
```

Complétez le main.

- (k) Modifiez la boucle principale pour afficher **Perdu!** lorsque le serpent sort ou qu'il se mord la queue (la méthode d'affichage est déjà fournie).
- (l) Il faut maintenant rajouter les pommes sur la grille. Écrivez les fonctions `cree_pommes`, `affiche_pommes`. Testez qu'elles fonctionnent bien.
- (m) Écrivez la fonction `mange_pommes`.
- (n) Testez votre fonction avec le programme suivant :

```
pommes = [(21, 16), (10, 10)]
serpent = [(20, 20), (21, 20), (21, 19), (21, 18)]

serpent = deplace_serpent(serpent, 'haut')
pommes = mange_pommes(pommes, serpent)
print(pommes)

serpent = deplace_serpent(serpent, 'haut')
pommes = mange_pommes(pommes, serpent)
print(pommes)
```

Vous devez obtenir :

```
[[ (21, 16), (10, 10) ]
[ (10, 10) ]
```

- (o) Rajoutez les pommes dans votre boucle principale et faites afficher **Gagné!** lorsqu'il ne reste plus de pommes (la méthode d'affichage est déjà fournie).
- (p) Faites en sorte que votre serpent grandisse de 3 cases lorsqu'il mange une pomme.